Examen Programación III - febrero 2023

- Tienes en ALUD la base para realizar el examen. Descarga el ZIP, descomprímelo y crea un proyecto en Eclipse con la estructura de carpetas suministrada. Trabaja en una ruta que puedas localizar fácilmente en el ordenador (por ejemplo el Escritorio).
- Para entregar tu respuesta a través de ALUD, comprime la carpeta raíz del proyecto que has creado en Eclipse, asegúrate de que tiene los ficheros que has editado y súbelo en la tarea de ALUD.
- Tiempo de examen: 3,5h.

Continuamos con nuestro sistema Easy Booking y ahora nos centraremos en el proceso de check-in para asignar asientos y generar las tarjetas de embarque. La nueva versión del código incorpora funcionalidad para realizar el check-in de las reservas que tiene un vuelo, esto es: mostrar la lista de personas que han realizado alguna reserva (recuerda que en cada reserva puede haber más de una persona), asignar un asiento disponible y por último almacenar los datos de la tarjeta de embarque en el repositorio de información (fichero CSV, fichero serializado y base de datos).

Al ejecutar esta nueva versión, verás que hemos añadido un botón "check-in" para cada uno de los vuelos. Al pulsar este botón se abre un cuadro de diálogo con una nueva tabla en la que podemos filtrar por datos de la persona o por código de reserva.

1. Tareas

Sobre este código, debes realizar una serie de tareas. Entre corchetes encontrarás la puntuación de cada tarea sobre 10 (+1 extra), las clases en las que debes añadir código y el número aproximado de líneas.

T1. Ficheros [1 punto] [AirAllianceService.java, 10-15 líneas]

- **T1.A Simula la impresión de las tarjetas de embarque creando un fichero de texto.**

  Implementa el método printBoardingPass() para generar un fichero de texto a partir de los datos de la tarjeta de embarque. El fichero se guardará en la carpeta resources/boardingPass/. El nombre del fichero debe ser: [CODVUELO]_[LOCALIZADOR]_[ASIENTO].txt (ej. *IB4353_ABN123_001.txt*). El formato del fichero tiene que ser el siguiente:

```
DATE-TIME: 2023/01/31 - 15:34          LOCATOR: ABN123
FLIGHT: IB4353                    FROM BIO TO BCN
PASSENGER: CLARK KENT             SEAT: 1
```

T2. Base de datos (BBDD)

El método loadFlights() obtiene la información de los vuelos que están almacenados en la bases de datos de la alianza StarAlliance pero esa base de datos NO permite gestionar el guardado y lectura de información de las tarjetas de embarque. Por ese motivo, debes realizar estas 3 tareas:

- **T2.A Crear la tabla BoardingPass** [0,5 puntos] [StarAllianceService.java, 5-10 líneas]:

  Implementa el método createBoardingPassesTable() para crear la tabla BoardingPass desde código. *La tabla debe crearse sólo si no existe*. La tabla tiene la siguiente estructura:

  - **LOCATOR** de tipo **TEXT**: localizador de una reserva [clave externa de la tabla Reservation(LOCATOR)].
  - **DATE** de tipo **TEXT**: fecha de creación (*se almacena el valor del Long, pero como texto*).
  - **PASSENGER** de tipo **TEXT**: nombre de la persona.
  - **SEAT** de tipo **INTEGER**: número de asiento.

  La clave primaria de esta tabla está formada por los campos LOCATOR y PASSENGER.

- **T2.B Cargar las tarjetas de embarque desde la BBDD** [1 punto] [StarAllianceService.java, 5-10 líneas]:

  Modifica el método loadReservationsDB() para que al recuperar los datos de las reservas se recuperen también los datos de las tarjetas de embarque asociadas a cada reserva.

- **T2.C Guardar una tarjeta de embarque en la BBDD** [0,5 puntos] [StarAllianceService.java, 5-10 líneas]:

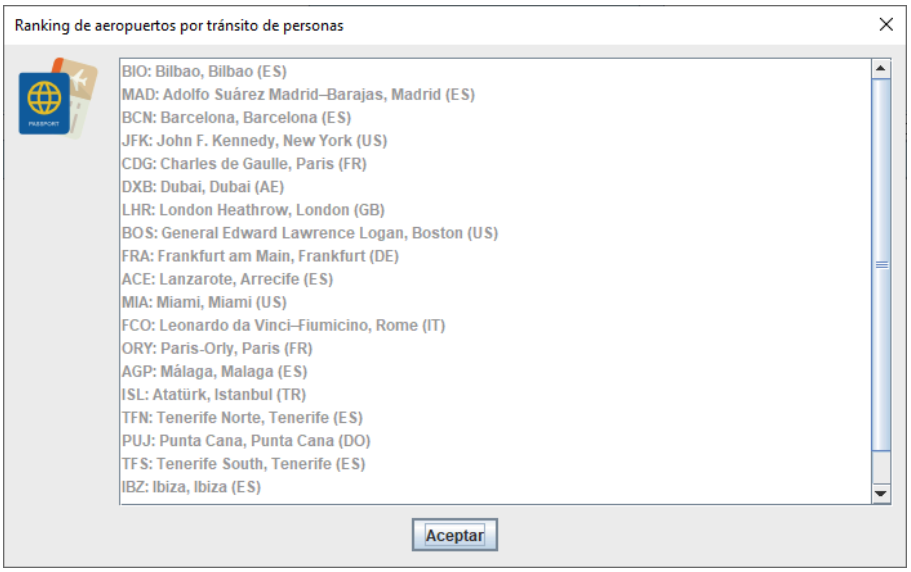
  Implementa el método storeBoardingPass(BoardingPass boardingPass) para almacenar una tarjeta de embarque en la BBDD.

T3. Colecciones [Statistics.java, 10-20 líneas]

La nueva versión del proyecto dispone de una clase para la generación de estadísticas a partir de la manipulación de datos de los vuelos y reservas.

- **T3.A Crea el ranking de tránsito de pasajeros por un aeropuerto** [1 punto]:

  Implementa el método List<Airport> getAirportTransitRanking(List<Flight> flights). Este método devuelve la lista ordenada de mayor a menor tránsito de pasajeros por un aeropuerto. El tránsito de pasajeros se calcula sumando el número de personas de las reservas de los vuelos que despegan o aterrizan en un aeropuerto. *La lista debe contener únicamente los aeropuertos por los que ha transitado al menos una persona*. *El cuadro de diálogo se carga automáticamente una vez generada la lista de aeropuertos.*



- **T3.B Crea el mapa de estadísticas de operaciones de las compañías aérea** [1 punto]:

  Implementa el método Map<Airline, List<Float>> getAirlineOperationsData(List<Flight> flights). Este método devuelve un mapa que indexa para cada aerolínea una lista con información acumulada de los vuelos operados por cada aerolínea. La lista contendrá los siguientes valores (en este orden):

  - Número total de ingresos de todas las reservas.
  - Número total de pasajeros (se calcula a partir del número de tarjetas de embarque).
  - Número de asientos de los aviones de todos los vuelos.

  El cuadro de diálogo se carga automáticamente una vez creado el mapa con los datos (ver en la siguiente página).



T4. JUnit [2 puntos] [TestStatistics.java, 20-25 líneas]

Crea un caso de prueba para validar el correcto funcionamiento de la implementación de los dos métodos de la clase Statistics.java. Para ello, en la clase TestStatistics.java en la carpeta de código fuente *test* añade una prueba para cada uno de los métodos de esta clase. Las pruebas deben validar lo siguiente:

- Para el método List<Airport> getAirlineOperationsData(List<Flight> flights):
  - Comprueba que la lista no es null.
  - Comprueba que la lista no está vacía.
  - Comprueba que cada elemento de la lista no es null.
- Para el método Map<Airline, List<Float>> getAirlineOperationsData(List<Flight> flights)
  - Comprueba que el mapa no es null.
  - Comprueba que el mapa incluye 12 aerolíneas.
  - Comprueba que ninguna clave ni valor son null.
  - Comprueba que todos los valores tienen un tamaño de 3.

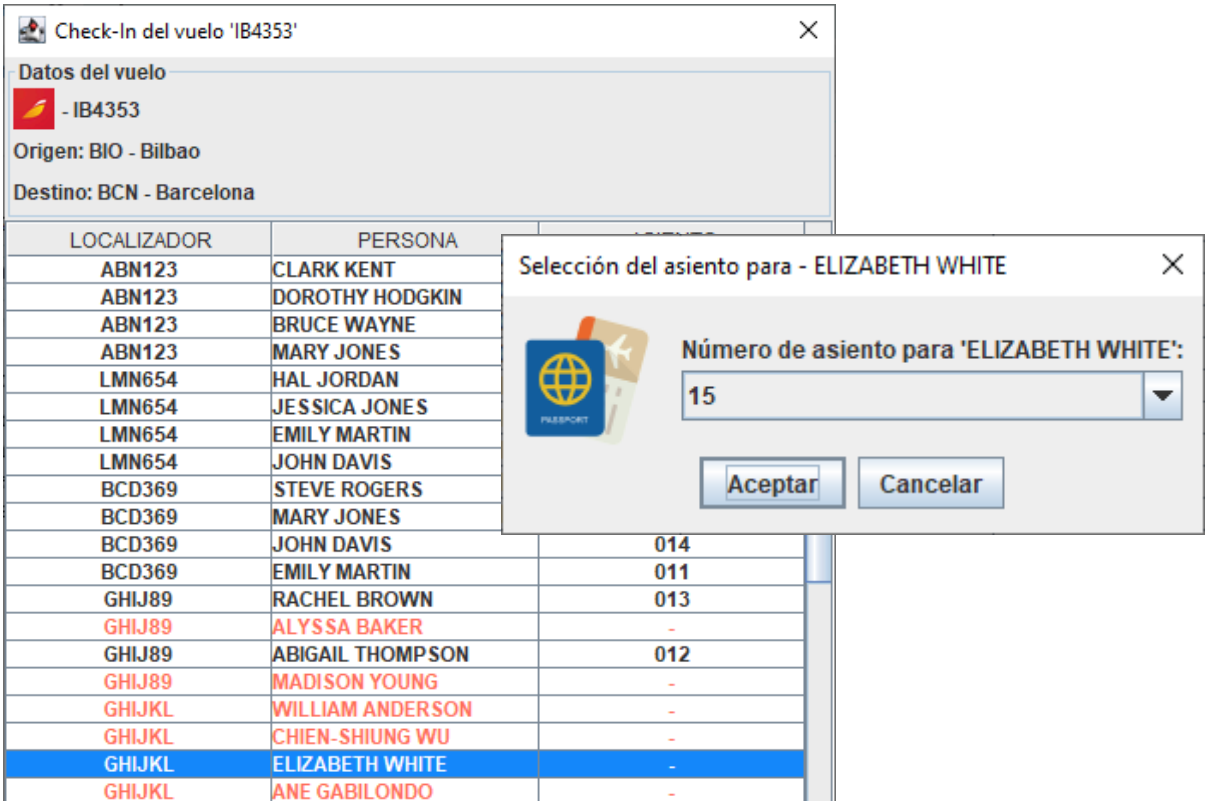
T5. Swing [2 puntos] (*Ver imagen en la siguiente página*)

- **T5.A Crear un renderer personalizado para la tabla de personas** [CheckInDialog.java, 25-30 líneas]

  Crea un Renderer personalizado para la tabla de personas que se muestra al pulsar el botón check-in. El Renderer debe realizar lo siguiente:
  - Poner en color RGB(255, 113, 91) el texto y fondo blanco de las filas sin asiento asignado.
  - Poner en color RGB(23, 137, 252) el fondo y texto blanco la fila seleccionada (tanto si tiene asiento asignado como si no lo tiene.
  - La columna LOCALIZADOR debe contener el localizador de la reserva y estar centrada.
  - El número de asiento debe aparecer centrado y con 3 dígitos (001 ó 045).

- **T5.B Pulsa F10 para asignar el asiento a una persona** [CheckInDialog.java, 25-30 líneas]

  Añade la funcionalidad necesaria para que cuando esté seleccionada una persona y se pulse la tecla F10 se abra un cuadro de diálogo para asignar el número de asiento (ver imagen). La lista desplegable de asientos sólo contendrá los asientos libres. Al pulsar el botón aceptar, se invocará el método createBoardingPass(Reservation reservation, String passenger, int seat) que se encuentra en la clase MainWindow.java.



T6. Recursividad [1,5 ó 2 puntos] [MainWindow.java, 10-15 líneas]

La nueva versión de la aplicación incorpora el botón "Búsqueda IDA y VUELTA". Este nuevo botón se activa al seleccionar el aeropuerto de origen. Cuando se pulsa este botón se abre un cuadro de diálogo para introducir una cantidad económica y a partir de ese importe realiza una búsqueda recursiva para obtener todos los itinerarios (secuencias de vuelos encadenadas) que permiten regresar al aeropuerto origen. Es decir, para cada itinerario, el aeropuerto origen del primer y último vuelo se corresponden con el vuelo seleccionado como origen. El coste de todos los vuelos de un itinerario debe ser siempre menor o igual a la cantidad introducida en el cuadro de diálogo.

Por ejemplo, saliendo desde Madrid con 340€ de máximo hay 34 itinerarios: 31 rutas de ida y vuelta con dos vuelos, y 3 rutas con tres vuelos (pasando por Ibiza y Bilbao).

Se pide que implementes el método *recursiveRoundtrip(...)*, puedes llevarlo a cabo de dos maneras. Cada una de ellas lleva asociada una puntuación diferente:

- Sacar por la consola cada itinerario (1,5 puntos)
- Generar un cuadro de diálogo similar al de la búsqueda recursiva del examen de convocatoria ordinaria (2 puntos)